ZAK MCP Gateway v0.4.1

How it works, in plain words: every technique, a picture of it, an everyday comparison, and the measured results.

96.8%
fewer input tokens over a whole 9-task agent run vs no gateway

−42%
vs v0.3 when the agent no longer has to search (pinned signatures)

18 → 11
model requests for the same work (each request re-sends everything)

−49% / −58%
vs your company lean server alone (heavy / light task, v0.3, 24/24 DB checks)

GitHub numbers measured on 6 Oct 2026 against the real GitHub MCP server (26 tools), public repo modelcontextprotocol/servers, o200k tokenizer, same calls in all setups. Lean numbers from your company benchmark (6 Oct, v0.3). Details and limits in section 8.

1. The problem in one minute

An AI agent uses tools (GitHub, files, databases, ticket systems) through **MCP**. The model is billed per **token**, roughly a short piece of a word. MCP wastes tokens in three places:

The manuals. Every tool's full JSON description goes with *every* request. GitHub alone is 3,600 tokens; a 227-tool company server is about 77,000.

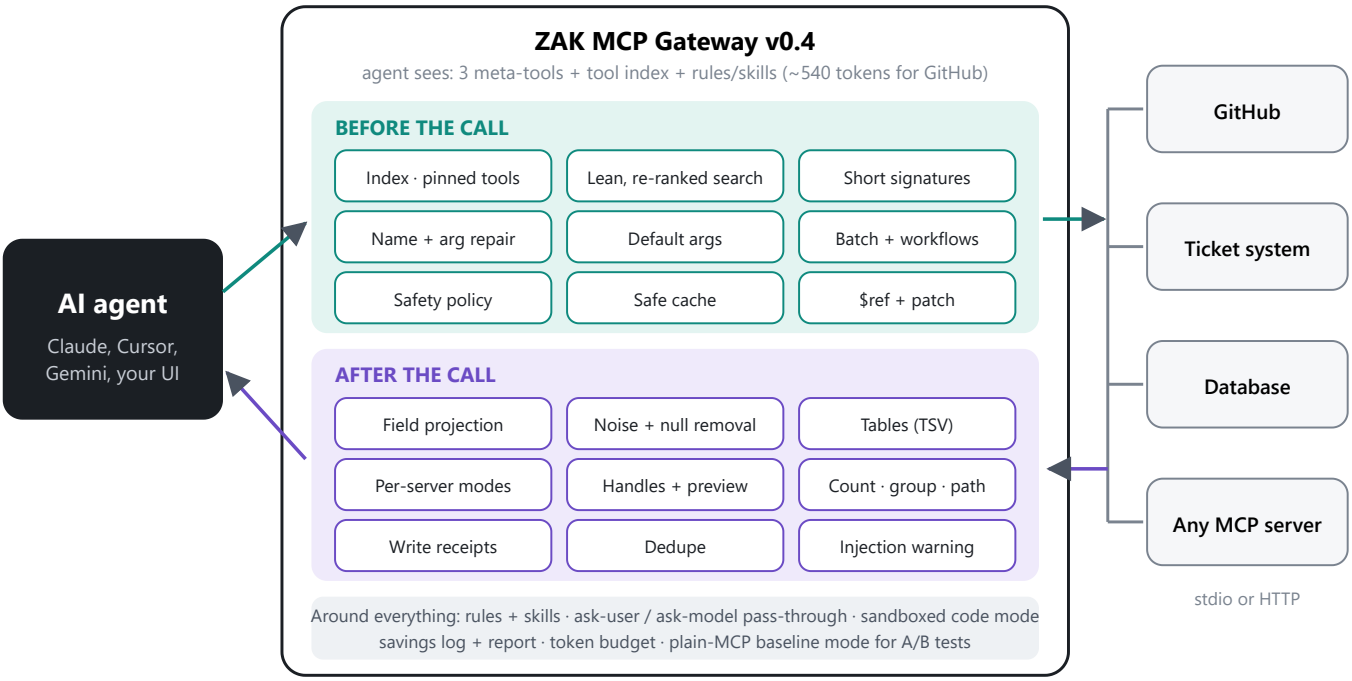
The results. Asking for 10 commits returns ~18,000 tokens of JSON (avatars, API links, IDs). The agent needed ~800.

The turns. Every tool call is another model request that re-sends the whole conversation. Extra searches and paging multiply all of the above.

> **Analogy:** you ask a hotel front desk for "a taxi". Before you can speak, they hand you the manual for every hotel service; when the taxi arrives they also give you the driver's employment file; and you have to walk back to the desk for every small question. The gateway is a good concierge: a one-page directory, a short answer, the file kept in the back office, and several errands handled in one trip.

2. The big picture

The gateway is one MCP server the agent connects to; all your real MCP servers sit behind it. Teal techniques shrink what goes **into** the model, purple ones shrink what comes **back**, and the grey strip makes any agent more precise and safe.


AI agent
Claude, Cursor, Gemini, your UI
ZAK MCP Gateway v0.4
agent sees: 3 meta-tools + tool index + rules/skills (~540 tokens for GitHub)
BEFORE THE CALL
Index · pinned tools
Lean, re-ranked search
Short signatures
Name + arg repair
Default args
Batch + workflows
Safety policy
Safe cache
$ref + patch
AFTER THE CALL
Field projection
Noise + null removal
Tables (TSV)
Per-server modes
Handles + preview
Count · group · path
Write receipts
Dedupe
Injection warning
Around everything: rules + skills · ask-user / ask-model pass-through · sandboxed code mode
savings log + report · token budget · plain-MCP baseline mode for A/B tests
GitHub
Ticket system
Database
Any MCP server
stdio or HTTP

3. What is new in v0.4: seven improvements

Your company test (gateway v0.3 in front of the lean server) showed where the remaining tokens go: **searches before writes** (46% of result tokens) and the **fixed cost of every turn** (tool list + instructions, re-sent each time). v0.4 targets exactly those. Every item is generic and off by default, so v0.4 with default settings behaves like v0.3.

#	Improvement	What it does (simply)	Real-world analogy
1	**Pinned signatures**	The one-line signatures of tools you choose (e.g. the 10–12 most-used write tools) are always in the instructions, so the agent calls them without searching.	The phone numbers you call daily, on a sticky note by the phone.
2	**Workflow tools**	A multi-step job written once in the config; the agent runs it with one call (see below). `npm run report` suggests workflows from repeated call sequences.	"The usual, please": the restaurant knows every step of the order.
3	**Cheaper fixed cost per turn**	Slim meta-tools (short descriptions, no parameter notes); sentences of a server's own instructions that only list tools are not repeated (the catalog has them); the compact `groups` catalog.	A pocket card instead of a printed brochure, handed over every visit.
4	**Leaner search replies**	Top hit as one line, other hits as names only (was ~85–300 tokens per search; now ~70–120).	The librarian points at one shelf instead of reading out the catalog.
5	**Smarter search ranking**	A tool whose *name* matches the request beats tools whose descriptions merely repeat a common word ("table"); "how many" finds count tools. GitHub accuracy unchanged (32/32 top-3).	Looking up "column" finds the column tool, not every page that says "table".
6	**History trimming** (agent host)	Old tool results in the conversation shrink to a short stub with their handle; the agent UI does this, and every agent host should.	Filing old paperwork instead of carrying it to every meeting.
7	**Prompt caching** (agent host)	Keep the tool list fixed for the session so the model provider's cache bills it at ~10%. The gateway keeps it fixed; the agent UI's "focused tools" mode changes it per question, so it trades caching for precision.	A season ticket: pay once, enter cheaply after.

Workflow tools in plain words

Without a workflow (build a table with 4 columns and a rule):
create table → *model turn* → add column → *turn* → add column → *turn* → … → add rule → *turn*
~10 model turns; each re-sends the tool list and the whole conversation.

With a workflow `wf__build_table`:
one call with the inputs (table name, columns, rule) → the gateway runs every step in order, passes values between them (the new table's id goes into each column step), and returns one short summary.
1 model turn.

```
"build_table": {
  "description": "Create a table with columns and a rule. Not for editing existing tables.",
  "input": { "properties": { "table": {"type":"string"}, "columns": {"type":"array"} },
             "required": ["table", "columns"] },
  "steps": [
    { "id": "table", "tool": "lean__ap_create_table",
      "arguments": { "name": "${input.table}" } },
    { "tool": "lean__ap_create_column", "forEach": "${input.columns}",
      "arguments": { "table": "${steps.table.id}", "field": "${item}" } },
    { "tool": "lean__ap_create_rule",
      "arguments": { "table": "${steps.table.id}", "when": "before_insert" } }
  ]
}
```

Safety stays the same	How
Writes need permission	One preview of all steps; nothing runs until the agent repeats the call with `confirm: true` (after asking the user).
Every step is checked	Each step goes through the normal safety rules, argument checks and cache, exactly like a single call.
Failures are precise	It stops at the first failing step and says which steps ran and which did not; a missing value never runs a step with an empty argument.
Nothing is lost	Every step's full reply is kept behind a handle; the summary is trimmed like any normal reply.
No surprises	Workflows are written (or approved from a `report` suggestion) by a person, never created by the agent itself; the normal tools stay available for unusual requests.

4. What was new in v0.3

v0.2 already made each answer small. Your company test showed the next problem: **extra turns**. Searches and re-paging added model requests, and each one re-sends the whole conversation. v0.3 attacks turns, keeps big data out of the conversation, and makes weaker models more precise.

Fewer turns

Call tools by name from the index; compact search; most-used tools listed up front; several calls in one batch; counts and groupings computed in the gateway.

Like a shopping list: one trip, not one trip per item.

Data stays in the gateway

Write receipts instead of full echoes; `$ref` passes stored data into the next call; patches edit it in place; one field in full by path.

Like forwarding a parcel without opening it.

Any agent, more precise

Your rules and skills reach every client; the downstream server can ask the user or the model through the gateway; a sandboxed code mode; helpers for weak models.

Like giving every new hire the same team handbook.

5. One request, step by step

Real example from the benchmark: "*show the last 10 commits of modelcontextprotocol/servers*", then "*how many open issues are pull requests?*"

1	**The agent reads the session instructions:** a tool index (`github: list_commits, list_issues, …`), pinned one-line signatures of the tools it will need, your rules and skill list. It calls `github__list_commits` directly, without searching.	in every request: ~550–650 tokens instead of 3,600
↓		
2	**Only if unsure, it searches.** The top hit comes back as one line, the others as names.	~70–120 tokens (v0.2: ~250–300)
↓		
3	**The gateway checks the call:** fixes small mistakes (`per_page` → `perPage`), adds configured default arguments, applies the safety rules and the cache.	0 extra turns
↓		
4	**GitHub answers** with pretty-printed JSON for 10 commits: avatars, ~20 API links each, signatures.	18,337 tokens raw
↓		
5	**The gateway distills it:** configured fields only, link noise and nulls removed, a table instead of JSON; the full original is kept as handle `r1` .	876 tokens to the model
↓		
6	**Counting without rows:** for "how many open issues are pull requests?" the agent asks `mcp_get_result({handle:"r2", grep:"/pull/", count:true})` and gets one line back.	55 tokens (v0.2 paged: 477)

What the model receives in step 5 (columns shortened for print):

```
sha       commit.message                                commit.author.name  commit.author.date    html_url
5abed86…  Merge pull request #4939 from …policy-to-main  Cliff Hall          2026-10-04T04:01:54Z  https://github.com/…
a207787…  chore: sync the bug report form with v2/main…  cliffhall           …                     https://github.com/…
… 8 more rows
[r1: default fields, noise keys removed; full data via mcp_get_result]
```

6. Every technique, with an everyday comparison

Technique	Where	What it does (simply)	Real-world analogy
Meta-tools	before	The agent sees 3 tools (search, call, get_result) instead of every server's tools.	One concierge desk instead of 40 counters.
Tool index new modes	before	Tool names in the instructions, as full names, families (`{create,get,list}_issue`), server list, or off. Sized for 20 or 200+ tools.	The lobby directory: detailed or by floor.
Hot signatures new	before	One-line signatures of the most-used tools (from the stats log) in the instructions, so the agent calls them without searching.	The popular dishes printed on the front of the menu.
Compact search new	before	BM25 search with synonyms; top hit as a full signature, the rest as name + summary. Servers can add their own words (ticket → incident).	A shortlist from the librarian, not the whole catalog.
Short signatures	before	A long JSON schema becomes one TypeScript-style line with parameter hints; too-complex schemas keep the full minified version.	A recipe card instead of the cookbook chapter.
Name + argument repair	before	Accepts `github::x` , plurals, `per_page` for `perPage` , `"5"` for 5; real mistakes return the signature.	A clerk who fixes a misspelled street.
Default arguments new	before	Per tool, the config fills in page size or server-side field selection when the agent leaves it out, so heavy data is never fetched.	A standing order: "the usual, small portion".
Batch calls new	before	Several calls in one turn, in order, one result line each. Writes are previewed first and run after one confirmation; stops at the first error and says where.	One trip with a full shopping list.
Safety policy	before	Read-only modes, allow/deny lists, writes need `confirm: true` ; read/write from MCP annotations, verbs, or per-tool overrides; unknown = write.	A teller asking for your signature.
Safe cache upgraded	before	Repeat reads from memory; a call becomes a write when its arguments say so (`writeIf`); servers on one backend share invalidation; per-server TTL or off; `fresh:true` .	Updating every copy of the menu when one changes.
Field projection	after	Only the fields asked for, or per-tool defaults; works through lists.	A highlighter on a long report.
Noise + null removal, tables	after	Drops API links, internal IDs, empty values; lists become tables when smaller.	Junk mail out; a spreadsheet, not a letter per row.
Per-server modes new	after	`full` for raw-JSON servers, `light` for already-compact servers (no reshaping, bigger limit), `off` .	Don't re-pack what is already packed.
Handles + preview	after	Big results stay in the gateway; the model gets the first rows and a handle (`r3`) to page, grep or pick fields.	A coat-check ticket.

Count, group, path new	after	`count`, `group_by`, `distinct`, `sort` on a stored result; `path` returns one field in full (e.g. a file's content).	Asking the clerk for the total, not every receipt.
Write receipts new	after	`{ok, id, …}` instead of the server's full echo; the echo stays behind a handle.	A receipt, not a copy of the whole order.
Dedupe new	after	A byte-identical repeat result becomes "unchanged, same as r5".	"Same as yesterday."
Injection warning	after	Results with text like "ignore previous instructions" are flagged as untrusted.	An "unverified sender" stamp.
`$ref` **+ patch** new	around	An argument can point at stored data (`{"$ref":"r3","path":"content"}`), even from another server, with find/replace edits. Fails loudly on a missing handle or an ambiguous edit.	Forwarding a parcel without opening it.
Rules + skills new	around	Your rules go into every session; skills are listed by name, loaded with `mcp_get_skill`, and suggested by search when they match.	The team handbook for every assistant, not just one.
Ask-user / ask-model pass-through new	around	When a downstream server needs to ask the user (elicitation) or the model (sampling), the gateway forwards it to the agent's client. Off by default.	Putting a call through to the right desk.
Sandboxed code mode upgraded	around	A short script calls several read tools; only the answer returns. Runs in a separate process with no file access, no child processes, a memory cap and a hard timeout. Off by default.	A locked workroom with one phone line.
Savings log, report, budget	around	Every call logged; `npm run report` shows savings, turns by kind and suggested default fields; optional session budget.	An electricity meter with a prepaid limit.
Plain-MCP baseline	around	`--passthrough` runs the same servers as plain MCP for fair A/B tests.	Weighing yourself on the same scale.
Pinned signatures v0.4	before	Chosen tools are always listed with one-line signatures, so writes need no search.	Daily numbers on a sticky note.
Workflow tools v0.4	around	A configured multi-step job in one call (`wf__name`), one confirmation, precise failure report, trimmed output.	"The usual, please."
Workflow suggestions v0.4	around	`npm run report` spots call sequences that repeat and prints a workflow skeleton for a person to review.	A waiter noticing you always order the same thing.
Slim meta-tools, lean search v0.4	before	Shorter tool descriptions every turn; one-line search hits; repeated tool inventories dropped.	A pocket card instead of a brochure.
Name-aware search ranking v0.4	before	Matches on a tool's name count more than words repeated in many descriptions.	Finding the shop by its sign, not by the street name.
Multi-table replies v0.4	after	A reply with several lists becomes several small tables instead of JSON.	One spreadsheet tab per list.

7. Helping weaker models (agent UI)

Small models (Gemini Flash, Claude Haiku) lose precision when they must pick among many tools, follow long rules, plan many steps and read big results. The rule is: **move decisions out of the model and into code.** The test agent UI now has four switches that do this:

Switch	What it does	Analogy
Preload tools + skill	Searches the first question up front and puts the best signatures and any matching skill in the prompt.	A waiter who brings water before you ask.
Focused tools	For each question, the 5 best-matching tools become real functions, so the model calls them directly instead of search-then-call.	Laying out only the tools for today's job.
Trim old results	Tool results from earlier turns are shortened in the history (with their handle), so they stop being re-sent in full every turn.	Filing old paperwork instead of carrying it around.
Verify the answer	IDs, SHAs and long numbers in the answer must appear in the tool results; otherwise the model gets one chance to correct itself.	A supervisor checking the figures before the report ships.

Mind the trade-off with prompt caching (#7): "focused tools" changes the tool list on every question, so the model provider cannot reuse its cached copy of the tool list. For strong models keep it off and rely on pinned signatures; for weak models the precision gain usually matters more.

Live result (Gemini, focused + trim + verify on): "last 3 commits" used 2 model calls and called the right tool with the right arguments on the first try. A harder counting question took 47k input tokens in 10 calls, against about 110k in 12 calls for a similar question on the old gateway. The run also exposed one real gap (a `path` meant per row), which is fixed and now has a test.

8. Measured results

GitHub MCP: no gateway vs v0.3 vs v0.4

Live, real GitHub MCP server (26 tools), public repo, 9 read-only tasks (the 8 from v0.3 plus a "repo overview"), same calls in every setup, o200k tokenizer. Command: `npm run bench:compare -- <older dist>`. v0.4 "tuned" = pinned signatures, groups catalog, slim meta-tools, one-line search, and a workflow for the overview.

No gateway v0.3 v0.4 tuned

solid = agent calls tools by name · outline = agent searches before each new tool · each panel has its own scale

Whole agent run: input tokens (all three)



Same, gateways only (zoomed)



Realistic pair: v0.3 has to search (101,171) · v0.4 has the signatures pinned, no search (58,715) → −42%

Model requests (each one re-sends tool list + conversation)



"No gateway" makes fewer requests (no searching) but each carries all 26 raw schemas and raw results, hence 20–30× more tokens.
v0.4 with default settings measured exactly like v0.3 (75,586 – 101,171): all v0.4 options are off by default.

Task	No gateway	v0.3	v0.4 tuned	What made the difference in v0.4
list_commits (10)	18,337	876 + 84	876 + 67	one-line search reply
list_commits (30), sha + message	61,589	1,280	1,280	same
list_issues (20 open)	37,846	909 + 157	909 + 118	leaner search
list_pull_requests (10)	27,375	839 + 223	839 + 120	leaner search
search_repositories (10)	3,470	485 + 144	485 + 71	leaner search
README	2,498	1,212 + 133	1,212 + 70	leaner search
list_commits again	18,337	876	876	cache
"How many of 30 open issues are PRs?"	52,148	909 + 55	909 + 55	count on the handle
Repo overview (commits, issues, PRs)	32,232 · 3 turns	753 · 3 turns	808 · 1 turn	workflow: one call instead of three

"+ n" = tokens spent on searching or follow-up reads (searches only happen when the agent searches). With pinned signatures, tuned v0.4 needs none, which is the 58,715 figure.

Your company lean server (measured by you, gateway v0.3)

Task	Lean alone	Lean + its batch tool	Gateway v0.3 → lean	Checks
Light: read and inspect (6 calls)	98,493	–	41,054 (groups: 39,486)	same rows, full JSX in one call
Heavy: build and verify	261,990	201,090	102,628	7/7 + 14 records each run
Heavy, searches removed (derived)	–	–	45,346	what pinned signatures (v0.4 #1) target

v0.4 has not been re-measured on the lean server yet. Expected from the derived figure and this GitHub run: heavy ~45–55k with pinned write signatures, lower with a build workflow.

Read these numbers honestly

- **Pinned signatures cost tokens on every turn** (+100 here). They pay off only by removing searches; pin the 10–12 tools used most, not everything.

- **Best case needs the model to cooperate** (call by name). With pinned signatures that is the natural behaviour, but only a live-model run proves it.
- **Workflows help multi-step jobs.** On the read-only GitHub test only the overview task could use one; build tasks (like your heavy task) gain much more.
- **Read-only benchmark.** Write receipts, batch writes, `$ref`/patch, trimming and skills are covered by 65 automated tests, not by this run.
- **Small setups can cost more** than plain MCP; the gateway pays off with many tools or big results.
- **Inside Claude Code** MCP is ~2.6% of all input, so whole-session savings there are small; MCP-heavy agents (your planned user agent) gain the most.
- **Tokens are not task success.** The o200k tokenizer stands in for Claude/Gemini. Real-model check so far: one Gemini A/B (20,806 → 2,522 input tokens).

9. Bugs found and fixed

Problem	Effect	Fix
v0.2		
Cache key built with `JSON.stringify(obj, keysArray)`	Every call hashed to the same key	Stable, sorted serialization; regression test
Caching never on; results double-encoded	No cache hits; results bigger than raw	Read/write classifier; return distilled text directly
`<YOUR_GITHUB_TOKEN>` placeholder	Overrode the real token → GitHub 401	Placeholders ignored; `${VAR}` secrets
`github::` vs `github__`; `per_page` vs `perPage`	"Tool not found"; setting silently ignored	One separator, tolerant lookup, argument repair
Agent UI "tokens saved" was a made-up formula	Misleading numbers	Real A/B with the model's own usage numbers
v0.3 (found by tests and live runs)		
Already-compact replies re-paged under one global limit	Extra turns on compact servers	Per-server limits and `light` mode
Raw paging cut long lines without saying so	The agent thought it had everything	Header says CLIPPED, and how to continue
Downstream server's own instructions dropped	Server guidance lost behind the gateway	Forwarded (capped), switchable per server
Grouped catalog format hid where the verb goes	Models would guess wrong names	`{create,get,list}_issue` notation + plural-tolerant lookup
Gemini sent `path` meaning "this field of each row"	Errors, 3 wasted turns in a live run	Falls back to a per-row field; test added
Answer verifier matched "1234" inside "INC0012345"	Wrong values could pass the check	Whole-token matching; test added
v0.4 (found by tests and the GitHub run)		
Workflow output carried raw step data	The overview workflow returned 28,384 tokens	Output uses trimmed views of each step; 808 tokens
Search ranked a description match above a name match	"add a field to a table" found the column tool 3rd	Name-aware re-ranking; 1st now; GitHub accuracy unchanged
A reply with several lists stayed JSON	Bigger multi-list replies	One small table per list

10. Plan coverage

Plan	Status	Notes
Build plan phases 1–4 (core, results, stats, safety)	done	Since v0.2.
Build plan phase 5 (benchmark)	partial	GitHub 3-way benchmark and live Gemini runs done; a 10–20 task real-model run over 3+ servers with success rates is still open.
Build plan phase 6 (advanced)	mostly	Sandboxed code mode, safe cache, tool preload, suggested default fields done; a cheap-model router belongs in the host app.
Economy plan: 14 ideas	all done	Call-first + hot signatures, compact search, batch, per-server limits, field by path, receipts, synonyms, safe cache, safer code mode, dedupe + stats, catalog modes, `$ref`, patch, suggested fields.
Extra ideas	done	Forwarded server instructions, default-argument pushdown, aggregation on handles, ask-user/ask-model pass-through, history trimming, live-model benchmarking.
Weak-model precision plan	in progress	Rules + skills serving, focused tools, verifier, skill preload, workflow tools done. Still open: evaluation set, name → ID resolution, escalation.
Next-steps plan (7 improvements)	done in v0.4	Pinned signatures, workflow tools + suggestions, cheaper turns, leaner search, search fix (generic); history trimming and prompt caching documented for agent hosts.

11. Use it in your own project (npm)

The gateway is published on npm as `zak-mcp-gateway`, so any project can use it without cloning this repository. Three steps: install it, describe your MCP servers in one config file, and register only the gateway with your agent.

> **In one picture:** the same concierge desk, now sold as a kit. Every building you run (every project) installs its own desk with one command, and guests (agents) are only ever given the desk's number.

1 · Install it in the project

```
npm install --save-dev zak-mcp-gateway     # recommended: the version is pinned in package.json
npx -y zak-mcp-gateway --help              # or try it without installing
npm install -g zak-mcp-gateway             # or install it as a global command
```

Needs Node.js 20 or newer.

2 · Create the gateway config

```
npx zak-mcp-gateway init                   # writes config/servers.json and prints the agent entry for step 3
```

`init` writes a small working config with one example server (GitHub) and never overwrites an existing file unless you add `--force` (`--config <path>` picks another place). Add your own servers under `mcpServers`, in the same shape as in any MCP client config. Every option is shown in `config/servers.example.json`, which ships inside the package. Write secrets as `${GITHUB_PERSONAL_ACCESS_TOKEN}` and set them in the environment. Add `.zak-gateway/` (the local savings log) to `.gitignore`.

3 · Register only the gateway with your agent

In `.mcp.json` (Claude Code), `claude_desktop_config.json` or `.cursor/mcp.json`, replace the direct entries for the servers the gateway now fronts with this one entry. `init` prints it with your real path already filled in.

```
{
  "mcpServers": {
    "zak-gateway": {
      "command": "npx",
      "args": ["-y", "zak-mcp-gateway", "--config", "/absolute/path/to/your-project/config/servers.json"]
    }
  }
}
```

Situation	What to change
Native Windows (not WSL)	Start npx through cmd: `"command": "cmd"`, `"args": ["/c", "npx", "-y", "zak-mcp-gateway", "--config", "C:/path/to/config/servers.json"]`. `init` prints this form on Windows.
You want exactly the version in your package.json	`"command": "node"` with `"args": ["/absolute/path/to/your-project/node_modules/zak-mcp-gateway/dist/cli.js", "--config", "..."]` : no npx lookup at start-up.
Secrets	In the host's `"env": { ... }` or your shell environment, never in `servers.json`.
Config path	Always absolute: not every host starts servers in your project folder. Relative paths inside the config resolve against the config file's folder.

Check and upgrade

```
npx zak-mcp-gateway report --config config/servers.json   # savings so far
npx zak-mcp-gateway --version                             # installed version
npm install --save-dev zak-mcp-gateway@latest             # upgrade, then restart the agent's MCP servers
```

Use it from code

```
import { loadConfig, GatewayServer } from "zak-mcp-gateway";

const gateway = new GatewayServer(await loadConfig("config/servers.json"));
await gateway.start();   // MCP over stdio, like the CLI
```

Works from `import` and `require`, with TypeScript types. The search index, signatures, safety policy, distillers and result store are exported too.

What the package contains

	Contents
ships	The compiled gateway with TypeScript types, the `zak-mcp-gateway` command, the full example config, README and LICENSE: about 86 kB to download.
never ships	Your `config/servers.json`, stats logs, tests, these documents and the agent UI. Publishing runs the full test suite first, then a clean build.

12. Versions and trying it yourself

Version	Date	Theme	Release note
v0.1	2–3 Oct 2026	First scaffold: two meta-tools, search, signatures, cache	`docs/versions/ZAK_Gateway_v0.1.pdf`
v0.2	4 Oct 2026	Build-plan upgrade: handles, trimming, safety, stats, A/B harness	`docs/versions/ZAK_Gateway_v0.2.pdf`
v0.3	5 Oct 2026	Economy upgrade: fewer turns, data stays in the gateway, any-client precision	`docs/versions/ZAK_Gateway_v0.3.pdf`
v0.4	6 Oct 2026	Seven improvements: pinned signatures, workflows, cheaper turns, leaner search	`docs/versions/ZAK_Gateway_v0.4.pdf`
v0.4.1	11 Nov 2026	Published on npm: `init`, `--version`, library exports; token behaviour unchanged	`docs/versions/ZAK_Gateway_v0.4.1.pdf`

```
npx zak-mcp-gateway init                       # in your own project: starter config + agent entry (section 11)
npm install && npm test                        # in this repo: 67 tests (unit, MCP round trips, features, CLI)
npm run test:github                            # live: CLI over stdio → GitHub MCP (no token needed for public repos)
npm run bench:compare -- <older dist dir>      # live benchmark: no gateway vs an older build vs this build (+ tuned)
npm run report                                 # savings, turns by kind, suggested default fields and workflows
node docs/src/versions/build-versions.mjs      # rebuild the per-version release notes
cd agent-ui && npm run dev                     # Gemini agent: A/B mode + weak-model switches
```